Ingeniería de requerimientos

Cuando hablamos de ingeniería de requerimientos, debemos tener en claro algunos conceptos que constituyen el desarrollo del *software*. Debemos identificar los diferentes tipos de requerimientos existentes, las diferentes formas de documentarlos, la forma de obtener la información a través de diferentes técnicas de elicitación y, por último, la explicación de los diferentes ciclos de vida.

- Ingeniería de requerimientos
- Concepto de requerimientos
- Documentando requerimientos
- Referencias

Ingeniería de requerimientos

Una cadena de cabañas permite realizar la reserva de habitaciones por medio de laweb. La persona interesada reservará un tipo de habitación para un período determinado que posee una tarifa que varía según la temporada (alta, media o baja). En el momento de la reserva, no deberá abonar nada, pero sí deberá hacerlo cuando la confirme, un mes antes del período elegido.La cadena tieneun catálogo con los tipos de cabañas con las que cuenta y las tarifas de acuerdo con la temporada, las cuales enviará dos veces por año a los clientes, quienes podrán consultarlas. En el momento de llegada al complejo, se asigna el número de cabaña.

La cadena de cabañas se encuentra en toda la Argentina y necesita, en cualquiera de las sucursales, poder realizar reservas y generar *check-in* y *check-out* de cada cliente.

Debido a un procesamiento inadecuado de los volúmenes de información, actualmente, las cabañascuentan con una tecnología desactualizada y, como consecuencia, tiene diferentes lenguajes de programación para procesar la información, lo cual influye en el funcionamiento de la página web.

El Equipo de Sistemas de las cabañas decide desarrollar los siguientes aspectos:

- el desarrollo de la página web debe ser rápido y se deben poder ver avances en poco tiempo, utilizando el ciclo de vida que se va a elegir.
- La página web debe poder actualizarse con diferentes versiones, a medida que la tecnología evolucione, y se debe utilizar PHP para su diseño.
- El software que necesita el hotel debe poder recibir la visita de mil clientes en dos horas.
- Se necesita un disco rígido para almacenar la información de, por lo menos, 1 *terabyte*.
- Se debe poder desarrollar un sistema de reservas que permita hacer un análisis de riesgos detallado, el cual deberá identificar los elementos que contiene el sistema de reservas y hacer

un relevamiento de la información obtenida, dentro de la empresa, a los responsables de esta área. Esto, a su vez, deberá complementar la información con lo obtenido de doscientos empleados que están relacionados con el Área de Reservas.

- Se necesitan registrar los requerimientos en un documento gráfico detallado y una herramienta gráfica.
- Se debe poder identificar cada interfaz de forma sencilla y que no falte ningún elemento.
- Las cabañas deben poder registrar, modificar, consultar, anular e imprimir las reservas
- Se debe poder desarrollar un sistema de facturación, para el cual se deberá contar con la información de veinticinco personas, lo cual permitirá facturar con un IVA del 25 % para clientes que vienen de Brasil.
- Es necesario contemplar los riesgos que comprende el sistema de facturación para saber qué medidas aplicar.
- Se necesita desarrollar un sistema de movimientos diarios de dinero, utilizando, para su construcción, el ciclo de vida secuencial, dada la claridad de los elementos que componen el sistema.
- Se viene desarrollando un sistema de cobranzas en el que se están probando los diferentes componentes del sistema con su código correspondiente.
- Para el sistema de *check-in*, se está terminando la versión 1.1, para poder probarla en el sistema operativo requerido. Para ello, se debe utilizar SQL como base de datos y se deben poder compartir recursos en la red que tiene el hotel.
- Una vez integrado el sistema de *check-in*, la cadena de cabañas debe contemplar que funcione bien ante cien solicitudes de *check-in* de 8 a 10 h, y que cada registro, consulta e impresión funcionen adecuadamente.
- Dentro del sistema de *check-in*, se debe poder realizar *testing* de fallas de datos, fallas de control, fallas de entrada y salida, fallas de interfaz, y fallas de gestión de almacenamiento.
- Es necesario que el sistema de reservas sea aprobado por los usuarios para verificar que funcione sin errores. También es importante que la empresa tenga una impresora láser a color que funcione con el sistema operativo Windows.

La cadena de cabañas cuenta con cinco sucursales por región: norte, sur, este y oeste.

A partir del problema de las cabañas, podemos ayudar al complejo a identificar los requerimientos de cada uno de los sistemas que va a desarrollar. Para ello, es fundamental desarrollar el concepto de requerimientos y las formas de documentarlos y administrarlos

CONTINUAR

Concepto de requerimientos

- Descripción de los servicios proporcionados por el sistema y las restricciones operativas.
- Los requerimientos reflejan las necesidades del cliente.
- El proceso de descubrir, analizar, documentar y verificar los servicios y restricciones se denomina ingeniería de requerimientos.

Uno de los principales problemas es no diferenciar los tipos de requerimientos.

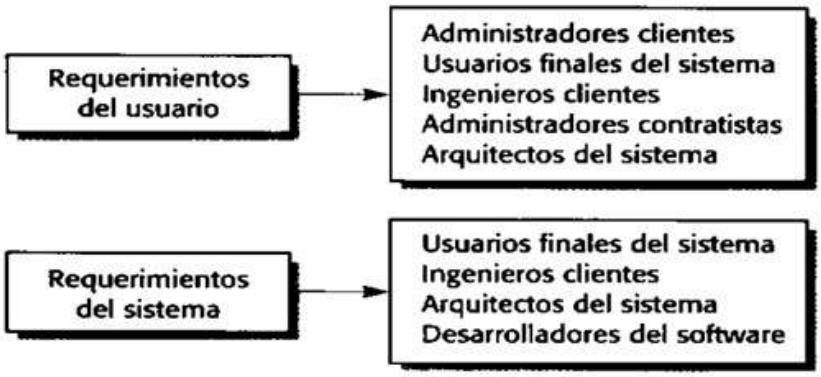
A continuación, se describen los tipos de requerimientos:

Requerimientos funcionales

Definen lo que debe hacer el sistema, puntualmente, ante determinadas entradas y, en algunas ocasiones, lo que no debe hacer. Dentro de los requerimientos funcionales, se da la siguiente clasificación:

- **requerimientos del usuario.** Se describen de forma abstracta. Son declaraciones en lenguaje natural y diagramas de los servicios de las funciones y restricciones del sistema.
- **Requerimientos del sistema:** definen, de forma detallada, las funciones, servicios y restricciones operativas del sistema (especificación funcional).

Figura 1: Requerimientos del usuario y el sistema



Fuente: Sommerville, 2005, p. 109.

Ejemplos de requerimientos funcionales

De acuerdo con Sommerville (2005), "…los requerimientos funcionales para un sistema *software* se pueden expresar de diferentes formas". (p. 110).

A continuación, se presentan "…algunos ejemplos de estos requerimientos funcionales para un sistema universitario de una biblioteca, denominado LIBSYS, utilizado por estudiantes y personal docente que solicitan libros y documentos de otras bibliotecas" (Sommerville, 2005, p. 110).

Figura 2: Ejemplos de requerimientos funcionales

Definición del requerimiento del usuario

1. LIBSYS controlará todos los datos requeridos por las agencias que licencian los derechos de autor en el Reino Unido y en otra parte.

Especificación de los requerimientos del sistema

1.1 Al hacer una petición de un documento del LIBSYS, el solicitante se presentará con un formulario que registre los detalles del usuario y de la petición hecha.
1.2 El formulario de petición del LIBSYS será almacenado en el sistema durante cinco años desde la fecha de la petición.
1.3 Todos los formularios de peticiones del LIBSYS se deben indexar por usuario, por el nombre del material solicitado y por el proveedor de la petición.
1.4 El LIBSYS mantendrá un fichero en el que se registrarán todas las peticiones que se han hecho al sistema.
1.5 Para el material donde se aplican los derechos de préstamo del autor, los detalles del préstamo serán enviados mensualmente a las agencias que licencian los derechos de autor que se han registrado con LIBYSS.

Fuente: Sommerville, 2005, p. 109.

Otros requerimientos que menciona Sommerville (2005) son los siguientes:

1. El usuario deberá tener la posibilidad de buscar en el conjunto inicial de la base de datos o seleccionar un subconjunto de ella.
2. El sistema deberá proporcionar visores adecuados para que el usuario lea documentos en el almacén de documentos.
3. A cada pedido, se le deberá asignar un identificador único (ID_PEDIDO), que el usuario podrá copiar al área del almacenamiento permanente de la cuenta. (p. 110).

Se puede observar que aquí también tenemos requerimientos de sistema.

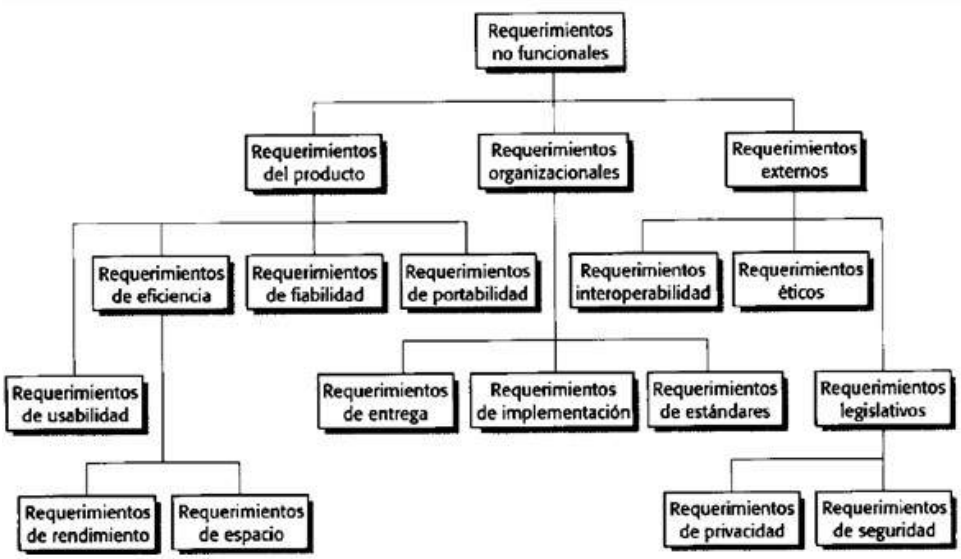
Requerimientos no funcionales

- Son restricciones de los servicios o funciones ofrecidos por el sistema. Estos incluyen las restricciones en el tiempo sobre el proceso de desarrollos estándares.
- No hacen referencia a las funciones.

- Se concentra en las propiedades (por ejemplo, la fiabilidad, los tiempos, el almacenamiento, etc.).
- Son más críticos que los requerimientos funcionales.
- No siempre se refieren al sistema de *software* a desarrollar; algunos pueden referirse al proceso que se utiliza en el desarrollo del *software*.
- La indicación de que el *software* debe ser desarrollado con una herramienta CASE particulares una especificación no funcional.
- Surgen den necesidades del usuario, restricciones económicas, políticas empresariales, interoperabilidad, *hardware*.

Ejemplos de requerimientos no funcionales

Figura 3: Ejemplos de requerimientos no funcionales



Fuente: Sommerville, 2005, p. 112.

Se puede observar que tenemos numerosos requerimientos no funcionales. Veamos algunos ejemplos de cómo redactar los requerimientos no funcionales:

- Se necesita que el sistema funcione en Windows10.
- Deberá soportar 10000 transacciones en el horario de 10 a 12 h.
- Tendrá que estar disponible las 24 h con copias de seguridad.
- Deberá tener un procesador de amplia velocidad de procesamiento, con las características específicas(características del procesador).
- Deberá tener una impresora con las características de XXXX.

Si analizamos los requerimientos no funcionales, podremos observar que son externos al sistema, tal como se puede notaren el ejemplo anterior.

Veremos en detalle algunas definiciones de requerimientos no funcionales:

1. **Requerimientos del producto**. Estos requerimientos especifican el comportamiento del producto. Algunos ejemplos son los requerimientos de rendimiento en la rapidez, de ejecución del sistema y cuánta memoria se requiere; los requerimientos de fiabilidad que fijan la tasa de fallos para que el sistema sea aceptable; los requerimientos de portabilidad, y los requerimientos de usabilidad

2. **Requerimientos organizacionales.** Estos requerimientos se derivan de políticas y procedimientos existentes en la organización del cliente y en la del desarrollador. Algunos ejemplos son los estándares en los procesos que deben utilizarse; los requerimientos de implementación, como los lenguajes de programación o el método de diseño a utilizar, y los requerimientos de entrega que especifican cuándo se entregará el producto y su documentación.

3. **Requerimientos externos.** Este gran apartado incluye todos los requerimientos que se derivan de los factores externos al sistema y de su proceso de desarrollo. Estos pueden incluir los requerimientos de interoperabilidad que definen la manera en la que el sistema interactúa con sistemas de otras organizaciones; los requerimientos legislativos que deben seguirse para asegurar que el sistema funcione dentro de la ley,

> y los requerimientos éticos. Estos últimos son puestos en un sistema para asegurar que será aceptado por sus usuarios y por el público en general. (Sommerville, 2005, p. 112).

Ejemplos de requerimientos no funcionales cuantificados

Tabla 1: Ejemplos de requerimientos no funcionales cuantificados

Rapidez	Transacciones procesadas por segundo. Tiempo de respuesta al usuario y a eventos Tiempo de actualización de la pantalla
Tamaño	K Bytes Número de chips de RAM
Facilidad de uso	Tiempo de formación Número de cuadros de ayuda
Fiabilidad	Tiempo medio entre fallos Probabilidad de no disponibilidad Tasa de ocurrencia de fallos Disponibilidad
Robustez	Tiempo de reinicio después de fallos Porcentaje de eventos que provocan fallos Probabilidad de corrupción de los datos después de fallos
Portabilidad	Porcentaje de declaraciones dependientes del objetivo Número de sistemas objetivo

Fuente: Sommerville, 2005, p. 114.

Para terminar, tenemos los requerimientos de dominio.

Requerimientos de dominio

Los requerimientos de dominio hacen referencia a una dirección IP. Por ejemplo:

- El sistema deberá tener el dominio .net (casa.net).
- El sistema deberá tener el dominio .edu (casa.edu.ar).

- El sistema deberá tener el dominio .org (casa.org).

Conclusión final

De acuerdo con lo desarrollado, podemos afirmar que tenemos los siguientes requerimientos:

1. requerimientos funcionales.

2. Requerimientos no funcionales.

3.Requerimientos de dominio.

Para que las cabañas puedan documentar los requerimientos obtenidos, se desarrollan los dos modelos que tenemos.

CONTINUAR

Documentando requerimientos

Para documentar los requerimientos, existen dos formas:

1. mediante un ERS/SRS-830, de forma detallada.
2. Mediante la realización gráfica de caso de uso.

1 Descubriendo un ERS/SRS

El estándar IEEE 830-1998 para el SRS(en inglés) o ERS (especificación de requerimientos de *software*) es un conjunto de recomendaciones para la especificación de los requerimientos o requisitos de *software*. Como producto final, obtiene la documentación de los acuerdos entre el cliente y el grupo de desarrollo, para cumplir, de esta manera, con la totalidad de exigencias estipuladas.

¿Quiénes leen el documento de especificación de requerimiento?

Clientes y usuarios

- Interesados en validar objetivos del sistema y descripción de alto nivel de la funcionalidad.
- Generalmente no están interesados en los requerimientos detallados del sistema.

Analistas (de sistemas, de requerimientos)

- Escribirán especificaciones de otros sistemas que interactúan con este.
- El SRS sirve más allá de la puesta en producción.

Desarrolladores (por ejemplo, arquitectos, diseñadores, programadores, etc.)

-Deben aplicarlos requerimientos.

Testers

-Deben determinar la satisfacción de los requerimientos.

Management

-Mideny controlan el proceso de desarrollo.

Otros lectores de SRS/ERS

- **Equipo de operaciones**

  – Deberán dimensionar equipos y procedimientos de rutina.

- **Equipo de soporte de usuario**

  – Desarrollan el plan de capacitación.

– Generan manuales de usuario.

– Procedimientos de soporte *online*.

- **Legales**

– Son aquellos encargados de firmar los contratos

- **Subcontratistas**
- **Entes reguladores**

Estructura de un SRS/ERS

- Proyecto: nombre del proyecto.
- Cliente: nombre del cliente.

Introducción

Describir la necesidad de crear el sistema y cuáles son sus objetivos.

¿Cuáles son los antecedentes y la historia detrás de este proyecto?

¿Cuáles son los acercamientos actuales a este problema?

Glosario de términos

- Términos técnicos del proceso que se emplean durante el proyecto a automatizar.
- Documentar las definiciones de términos y acrónimos permite a otros documentos ser más concisos.

Definición de requerimientos

- Definición de las necesidades del problema a solucionar, mencionando los aspectos esenciales, dando prioridades, funciones, características y restricciones.

Definición de requerimientos funcionales

- Describen la interacción entre el sistema y su ambiente, deforma independiente a su aplicación.
- El ambiente incluye al usuario y cualquier otro sistema externo con el cual interactúa el sistema a desarrollar.
- Definir los servicios que se proporcionarán.

Definición de requerimientos no funcionales

- Define las limitantes del sistema y el proceso de desarrollo.

Requerimiento del sistema (factibilidad técnica)

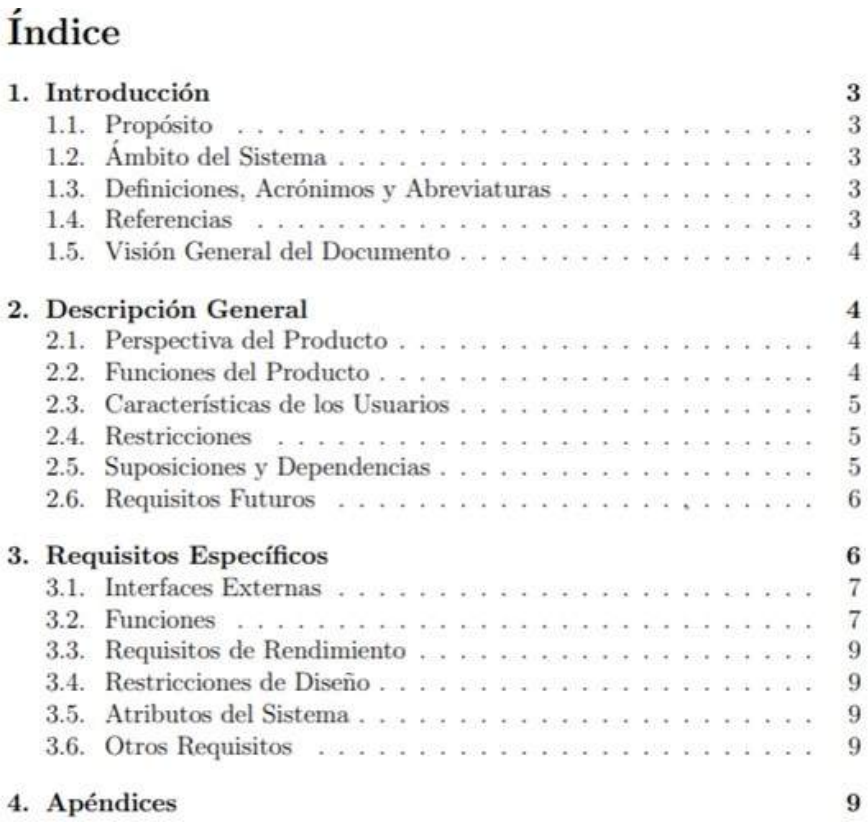
- Descripción de la plataforma de *hardware* del sistema.
- Especificación del *software* del sistema.
- Interfaces de aplicación del programa (API) que deben incluirse.
- Requerimientos de importación y exportación de datos.
- Requerimientos de las bases de datos o archivos.

Evolución del sistema

- Definir las suposiciones fundamentales en las cuales el sistema se basa y en las que se anticipan los cambios.

Estructura de un documento SRS/ESR-IEEE 830-1998

Figura 4: Estructura de un documento SRS/ESR-IEEE 830



Índice

1. **Introducción** 3
   - 1.1. Propósito . . . 3
   - 1.2. Ámbito del Sistema . . . 3
   - 1.3. Definiciones, Acrónimos y Abreviaturas . . . 3
   - 1.4. Referencias . . . 3
   - 1.5. Visión General del Documento . . . 4
2. **Descripción General** 4
   - 2.1. Perspectiva del Producto . . . 4
   - 2.2. Funciones del Producto . . . 4
   - 2.3. Características de los Usuarios . . . 5
   - 2.4. Restricciones . . . 5
   - 2.5. Suposiciones y Dependencias . . . 5
   - 2.6. Requisitos Futuros . . . 6
3. **Requisitos Específicos** 6
   - 3.1. Interfaces Externas . . . 7
   - 3.2. Funciones . . . 7
   - 3.3. Requisitos de Rendimiento . . . 9
   - 3.4. Restricciones de Diseño . . . 9
   - 3.5. Atributos del Sistema . . . 9
   - 3.6. Otros Requisitos . . . 9
4. **Apéndices** 9

Fuente: Estándar IEEE 830.

---

2 Caso de uso

Diagrama de casos de uso

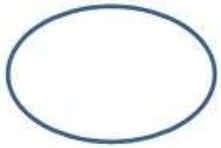
Un diagrama de casos de uso muestra actores, casos de uso, paquetes de casos de uso y sus relaciones. Los diagramas de casos de uso pueden organizarse en paquetes de casos de uso y

mostrar solo lo que es relevante con un paquete particular.

Componentes de un caso de uso

El nombre del caso de uso debe colocarse dentro del círculo.

Figura 5: Caso de uso

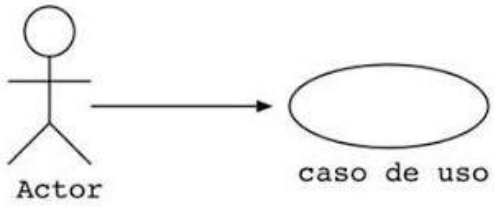


Fuente: elaboración propia.

Un caso de uso registra el contrato entre los involucrados del sistema acerca de su comportamiento en varias circunstancias y organiza por los actores involucrados.

Identificando actor

Figura 6: Identificar el actor



Fuente: elaboración propia.

Un actor de uso es cualquier cosa que intercambia datos con el sistema. Puede ser un usuario externo o un *software*. Un actor representa a una clase, y varios usuarios pueden desempeñar el mismo papel.

Figura 7: Cliente, comunicación, caso de uso



Fuente: elaboración propia.

Como se puede observar, el actor es el **cliente** y el caso de uso se llama **Realizar consulta**. Luego, deberemos definir la interfaz que indique cómo está compuesta «**Realizar consulta**» en otro documento, es decir, la interfaz en la pantalla gráfica en la que el cliente va a ingresar los datos para realizar la consulta.

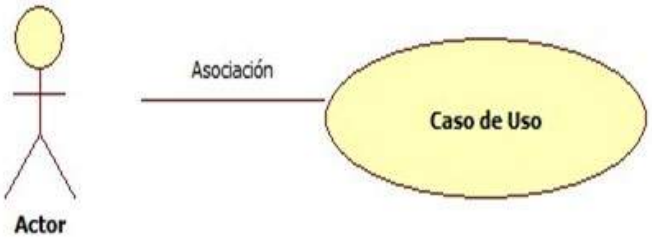
Es fundamental tener en cuenta que el nombre del caso de uso empieza con un verbo en infinitivo(en este caso, Realizar consulta).

Otros ejemplos de caso de uso pueden ser los siguientes:

- Registrar cliente.
- Registrar modificación.
- Anular cliente.

En ninguna circunstancia, el nombre del caso de uso puede ser Ingresar DNI, dado que el ingreso del DNI del cliente es parte de la interfaz para realizar la consulta.

Figura 8: Asociación

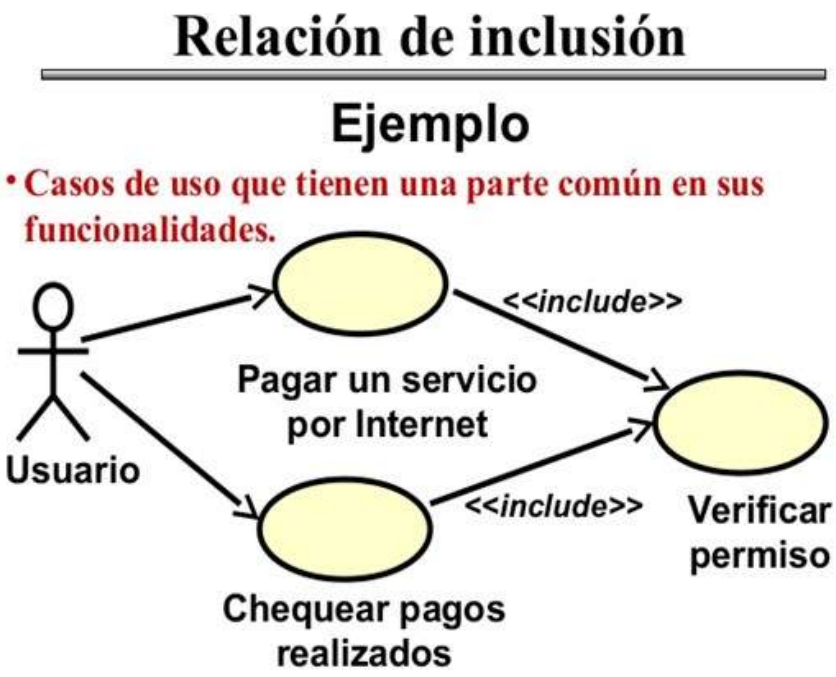


Fuente: elaboración propia.

La línea que une al actor con el caso de uso se llama Asociación y se debe respetar porque es parte de la metodología.

Descubriendo el include

Figura 9: Relación de inclusión



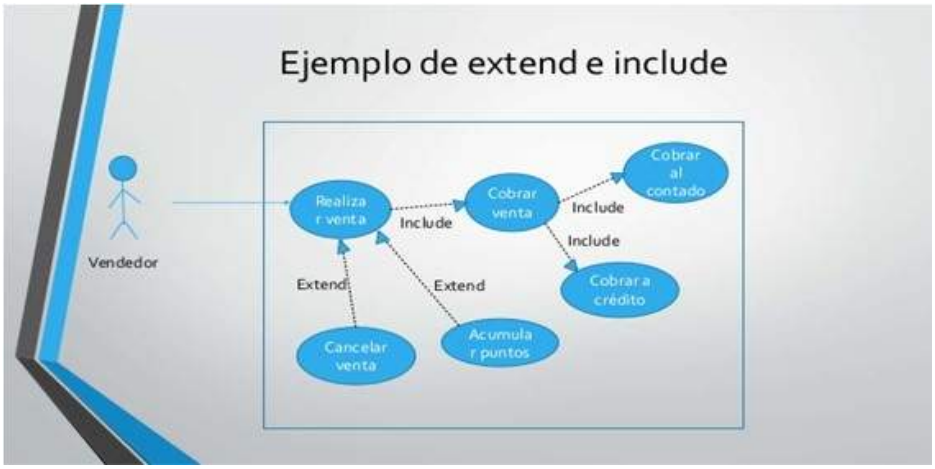
Fuente: Pascual Mariño, 2015, https://bit.ly/3yrYSe9.

El include permite una relación estrecha y directa con el caso de uso. Se puede observar que el include está relacionado con dos casos de usos por separado: 1) pagar un servicio por internet y 2) revisar pagos realizados.

Observen que la línea tiene la dirección hacia el caso de uso Verificar permiso. No permite continuar el proceso sino verificó el permiso.

Veamos otro ejemplo.

Figura 10: Ejemplo de extend e include



Fuente: Aravena, 2016, https://bit.ly/2TdrU0Q.

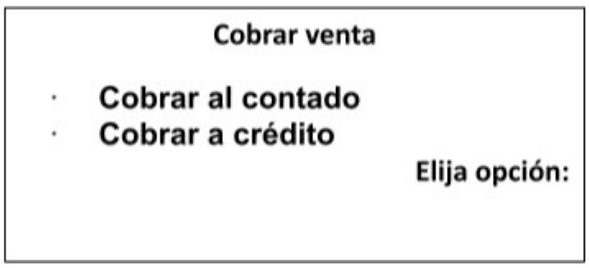
Podemos observar que, del caso de uso Realizar venta, sale un include que es obligatorio, llamado Cobrar venta.

Cobrar venta será una interfaz (pantalla) que tendrá dos include:

- Cobrar al contado
- Cobrar a crédito.

A continuación, encontrarás un ejemplo.

Figura 11: Ejemplo de interfaz



Fuente: elaboración propia.

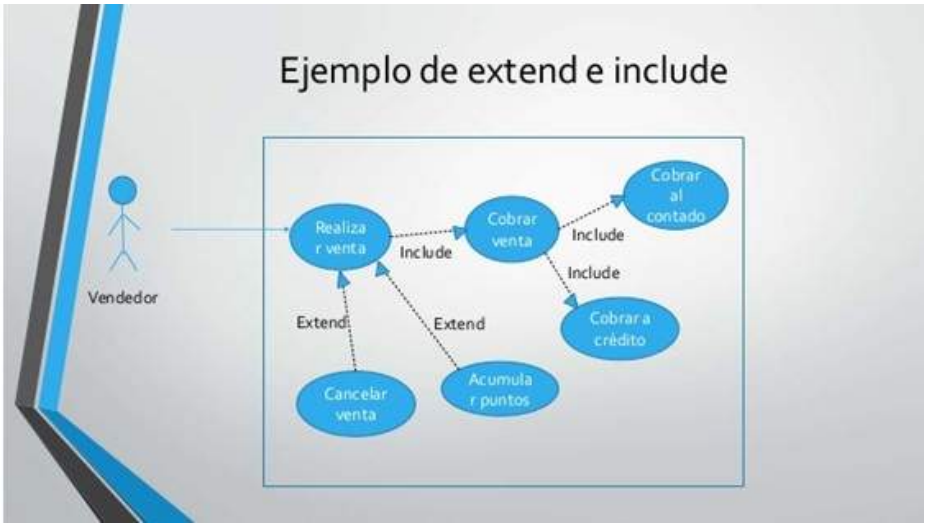
Por lo tanto, puede establecerse que el caso de uso Cobrar venta tiene dos include relacionados: Cobrar al contado y cobrar a crédito.

Cobrar al contadoy cobrar a crédito van a habilitar diferentes pantallas para terminar con el proceso de cobranza.

Es necesario tener en cuenta que cada caso de uso permite construir una interfaz. Si no se puede construir una interfaz, el caso de uso está mal confeccionado. El include significa que es necesario y obligatorio entrar a una de las dos opciones para terminar la operación de venta. Sino se entra a una de las dos opciones que figuran en el include, el proceso de venta no se puede terminar.

Descubriendo el extend

Figura 12: Ejemplo de extend e include



Fuente: Aravena, 2016, https://bit.ly/2TdrU0Q.

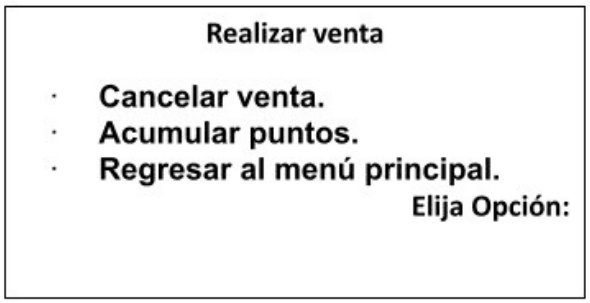
Considerando el mismo ejemplo, el **extend** implica que va a haber dos nuevos casos de usos disponibles **sin la obligación** de entrar para terminar el proceso.

En el ejemplo de la figura 12, se puede observar que tenemos dos extend dependiendo de Realizar venta:

- Cancelar venta (caso de uso).
- Acumular puntos (caso de uso).

Veamos la interfaz de extend.

Figura 13: Ejemplo de interfaz



Fuente: elaboración propia.

Como se puede observar, en la interfaz **no** es obligatorio entrar a las opciones. Está la posibilidad de volver al menú principal, lo cual establece una gran diferencia con respecto a un include.

Es necesario tener en cuenta que cada caso de uso permite construir una interfaz. Si no se puede construir una interfaz, el caso de uso está mal confeccionado.

Otro ejemplo

Figura 14: Caso de uso múltiple



Fuente: [imagen sin título sobre caso de uso múltiple], s.f., https://bit.ly/3bMDOVS.

Se puede observar que el caso de uso Pagar un servicio por internet está relacionado, de forma obligatoria, con dos include:

- Verificar permiso.
- Redefinir deuda.

Se puede observar que el caso de uso Pagar un servicio por internet está relacionado, de forma optativa, con un **extend**.

Reportar incongruencias

Para terminar, es fundamental tener en cuenta siempre lo siguiente:

1. El nombre del caso de uso empieza con un verbo en infinitivo.
2. El include indica que es obligatorio entrar en la interfaz correspondiente para terminar el proceso.

3. El extend indica que no es obligatorio entrar a la interfaz correspondiente para terminar el proceso.

4. Es necesario tener en cuenta que cada caso de uso permite construir una interfaz. Si no se puede construir una interfaz, el caso de uso está mal confeccionado.

5. A partir del caso de uso, se realiza el siguiente proceso:

   5.1 realizar caso de uso.

   5.2 Realizar la interfaz de cada caso de uso. El contenido de la interfaz se explica detalladamente en el *template*.

   5.3 Las interfaces permiten desarrollar el diagrama de clases.

   5.4 A partir del diagrama de clases, realizar el diagrama de secuencia.

Haremos un repaso de los principales puntos de la lectura.

El requerimiento de rendimiento es un requerimiento funcional.

- ◯ Falso.
- ◯ Verdadero.

SUBMIT

El SRS-830 es un documento que permite documentar solamente requerimientos funcionales.

- ◯ Falso.
- ◯ Verdadero.

SUBMIT

¿Puedes identificar los elementos que componen un caso de uso?

- ☐ Actor.
- ☐ Asociación.
- ☐ Include.
- ☐ Comunicación.
- ☐ Herencia.

SUBMIT

El SRS-830 tiene una etapa en la que se define la perspectiva del producto. ¿Puedes identificar dicha etapa en la siguiente lista?

- ◯ Descripción general.
- ◯ Requerimientos específicos.
- ◯ Introducción.
- ◯ Apéndices.

SUBMIT

CONTINUAR

Referencias

[Imagen sin título sobre caso de uso múltiple], (s. f.). Recuperado de https://slidetodoc.com/curso-de-uml-actividad-2-diagramas-de-casos/.

Aravena, S. (2016). *Gestión de proyectos informáticos*. Recuperado de https://www.slideshare.net/SergioAravenaVidal/caso-de-uso-e-historia-de-usuario

Estándar IEEE 830. (1998). Recuperado de https://www.fdi.ucm.es/profesor/gmendez/docs/is0809/ieee830.pd

Pascual Mariño, J. (2015). *Diagrama de casos de uso del negocio y del sistema*. Recuperado de https://es.slideshare.net/jerrypascualmarino/ejemplo-48058523

Sommerville, I. (2005). *Ingeniería de Software* (7.ma. ed.). Madrid, España: Editorial Pearson.

CONTINUAR